Document number	P3216R0
Date	2024-04-01
Audience	LEWG, SG9 (Ranges)
Reply-to	Hewill Kang <hewillk@gmail.com>

`views::slice`

Abstract

This paper proposes the Tier 1 adaptor in P2760: `views::slice` to improve the C++26 ranges facilities, which is a simple composition of `views::take` and `views::drop`.

Note that this is the first range adaptor in the standard that accepts *two* arguments, namely *`begin_index`* and *`end_index`* to specify the slice interval [*`begin_index`*, *`end_index`*).

Revision history

R0

Initial revision.

Discussion

Slice is very common operation in other languages, there is also a similar view in range/v3, `views::slice`, which returns a *slice* of the original range bounded by two indices.

Unfortunately, due to its relatively minor importance at the time, `slice` was only classified as Tier 3 in P2214. However, as the three sliding window brothers: `chunk`, `slide`, and `stride` entered C++23, `slice`, which was once neglected, entered the public eye again.

Although it is classified in the `take`/`drop` family in P2760, the author believes that it can also be regarded as a window-related utility to a certain extent because it cooperates well with the slding family. In addition, `slide` can provide a more comfortable and generic way to obtain a *slice* of a range rather than `subrange` and `counted`, while the latter two still have certain limitations when applied to non-`forward_range` or non-`sized_range`s.

Given the above, the author believes that it is time to bring `slice` into C++26.

Design

Should we introduce a new `slice_view` class?

Nope.

As stated in P2214: "`slice(M, N)` is equivalent to `views::drop(M) | views::take(N - M)`, and you couldn't do much better as a first class view. range-v3 also supports a flavor that works as `views::slice(M, end - N)` for a special variable `end`, which likewise be equivalent to `r | views::drop(M) | views::drop_last(N)`."

This means that `slice(M, N)` can simply be a trivial alias of the latter two, and author believes that such a design has fully accommodated the current desires.

First, the implementation of `slice` involves advancing to the beginning of the slice and calculating the end, which perfectly matches matches what `drop` and `take` is currently doing. If this is the case, the author sees no reason not to compose them.

Second, those two have certain specializations for return types in order to avoid template explosion, composing them means directly inheriting existing optimizations. In other words, `span{v} | slice(1, 10) | slice(3, 5)` will still produce a `span` containing the proper slice. And if there are any improvements for `drop` and `take` in the future, `slice` can automatically benefit.

It is worth noting that since `drop` and `take` support output ranges, which makes `slice` also support output ranges. There is no worthwhile reason for not supporting it.

Should the second argument be *size*?

Nope.

An alternative design for `slice` is to accept a starting index and a *size*. However, the author prefers to use the end index as the second parameter since this is intuitive and consistent with other language syntaxes such as Python: `a[1:3]` and Rust: `&a[1..3]`, etc.

What if out of range or *`end_index`* is smaller than *`begin_index`*?

Since `slice` is a composition of `drop` and `take`, it also inherits their handling out-of-range indexes. That is to say, `iota(0, 5) | slice(3, 9)` will get `iota(3, 5)` and `iota(0, 5) | slice(7, 9)` will get an empty range. This is fine because it does conform to current standard behavior. Otherwise, what we need is probably a `slice_exactly` (via `drop_exactly` and `take_exactly`?)

Another thing to note is that both `drop_view` and `take_view` have the *Preconditions* that `count >= 0`, which makes `slice(r, -1, 1)` or `slice(r, 2, 1)` UB if it eventually constructs `drop_view(r, -1)` or `take_view(r, -1)`; if it goes into specialized branches, then constructors of other views such as `iota_view` or `repeat_view` also have similar *Preconditions* to guarantee valid ranges.

Do we need to support special variable *`end`*?

Nope.

Although range/v3 supports a special variable *`end`* for universally denoting the end index of a range, making it possible to spell it like `views::slice(M, end - N)`, the author believes that such use case is too specific and may become less and less necessary with the subsequent introduction of `drop_last`.

Should we provide a variant of `slice(1, 10, stride)`?

Nope.

Other languages such as Python also support stride as the third parameter of slice such as `[1:10:3]`, which can be done by declaring `slice(M, N, P) as slice(M, N) | stride(P)`. However, the authors do not see much value in providing this variant.

First, `slice(M, N) | stride(P)` can express the meaning of the third argument `P` more explicitly than `slice(M, N, P)`. In addition, `views::slice(M, N, P)` may be confused with similarly named classes in the standard, such as `std::slice` and `std::strided_slice`, whose second parameter has completely different meanings, representing *size* and *extent* respectively instead of ending index. There is really no need to introduce another layer of confusion.

Implementation experience

The author implemented `views::slice` based on libc++. The details are relatively simple in about 20 lines, see here.

Proposed change

This wording is relative to N4971.

1. Add a new feature-test macro to 17.3.2 [version.syn]:

```
#define __cpp_lib_ranges_slice 2024XXL // freestanding, also in <ranges>
```

2. Modify 26.2 [ranges.syn], Header <ranges> synopsis, as indicated:

```
#include <compare>              // see [compare.syn]
#include <initializer_list>     // see [initializer.list.syn]
#include <iterator>             // see [iterator.synopsis]

namespace std::ranges {
  […]
  namespace views { inline constexpr unspecified drop_while = unspecified; }        // freestanding
```

```
  // [range.slice], slice view
  namespace views { inline constexpr unspecified slice = unspecified; }              // freestanding
  […]
}
```

3. Add **26.7.? Slice view [range.slice]** after 26.7.13 [range.drop.while] as indicated:

-1- A slice view presents a view of elements from position *N* up to but not including position *M* of the original view. The resulting view is empty if the original view has fewer than (*N*+1) elements, or all elements starting from position *N* if it has fewer than *M* elements.

-2- The name `views::slice` denotes a range adaptor object ([range.adaptor.object]). Let `E`, `F` and `G` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` and `decltype((G))` do not model `convertible_to<D>`, `views::slice(E, F, G)` is ill-formed. Otherwise, the expression `views::slice(E, F, G)` is expression-equivalent to `views::take(views::drop(E, F), static_cast<D>(G) - static_cast<D>(F))`, except that `F` is evaluated only once.

-3- [*Example 1*:

```
auto ints = views::iota(0);
auto fifties = ints | views::slice(50, 60);
println("{} ", fifties); // prints [50, 51, 52, 53, 54, 55, 56, 57, 58, 59]
```

— end example]

References

[P2760R1]
Barry Revzin. A Plan for C++26 Ranges. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r1.html

[P2214R2]
Barry Revzin. A Plan for C++23 Ranges. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2214r2.html